

CO1

FLOWGORITHM FLOWCHART AND PSEUDOCODE ASSIGNMENT



Name	M. Yogeshwaran
Register No.	192524363
Course Name	Java Programming
Course Code	CSA0924

Submitted as part of the Java Programming coursework

Question 1: BMI Calculator

Aim: Read the weight in kilograms and height in metres. Calculate the BMI and classify it as Underweight, Normal, Overweight, or Obese.

BMI Classification Table

BMI Range	Category
BMI < 18.5	Underweight
$18.5 \leq \text{BMI} < 25$	Normal
$25 \leq \text{BMI} < 30$	Overweight
BMI ≥ 30	Obese

Flowchart

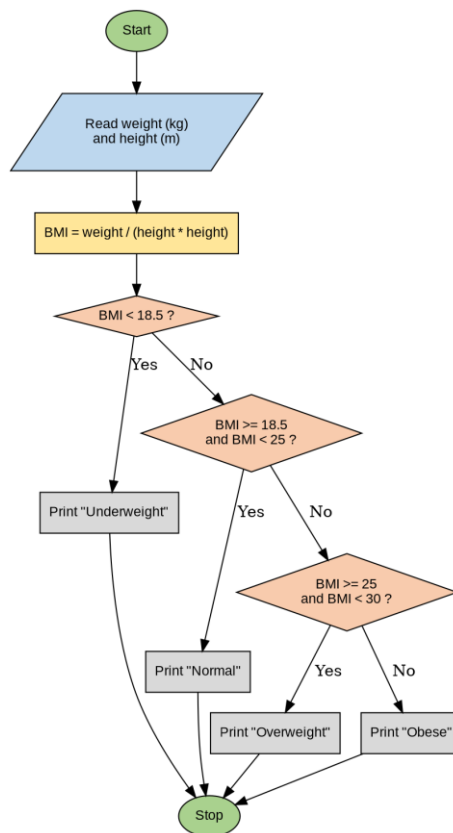


Figure 1.1: Flowchart for BMI Calculation and Classification

Pseudocode

```

BEGIN
  READ weight
  READ height
  bmi = weight / (height * height)
  IF bmi < 18.5 THEN
    PRINT "Underweight"
  ELSE IF bmi >= 18.5 AND bmi < 25 THEN
    PRINT "Normal"
  ELSE IF bmi >= 25 AND bmi < 30 THEN
    PRINT "Overweight"
  ELSE
    PRINT "Obese"
  END IF
END

```

Question 2: Check Whether an Array is Sorted (Ascending)

Aim: Read N values into an array and check whether the array is sorted in ascending order.

Flowchart

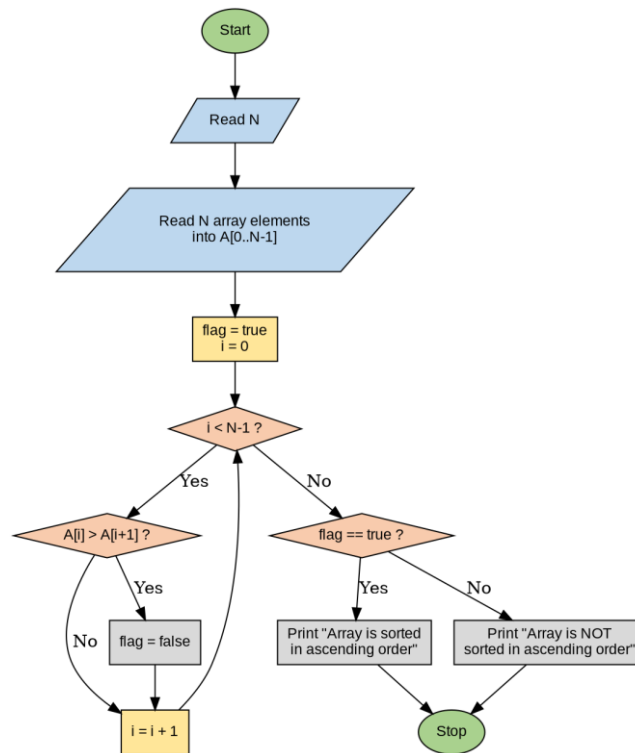


Figure 2.1: Flowchart to Check Ascending Order of an Array

Pseudocode

```

BEGIN
  READ N
  FOR i = 0 TO N-1
    READ A[i]
  END FOR

  flag = TRUE
  FOR i = 0 TO N-2
    IF A[i] > A[i+1] THEN
      flag = FALSE
      EXIT FOR { array not sorted, break early }
    END IF
  END FOR

  IF flag == TRUE THEN
    PRINT "Array is sorted in ascending order"
  ELSE
    PRINT "Array is NOT sorted in ascending order"
  END IF
END
  
```

Question 3: Food Ordering System (Regular vs Premium Customer)

Aim: Design a food-ordering flowchart with common order details and different delivery charges for Regular and Premium customers, and relate the design to the concept of inheritance.

Flowchart

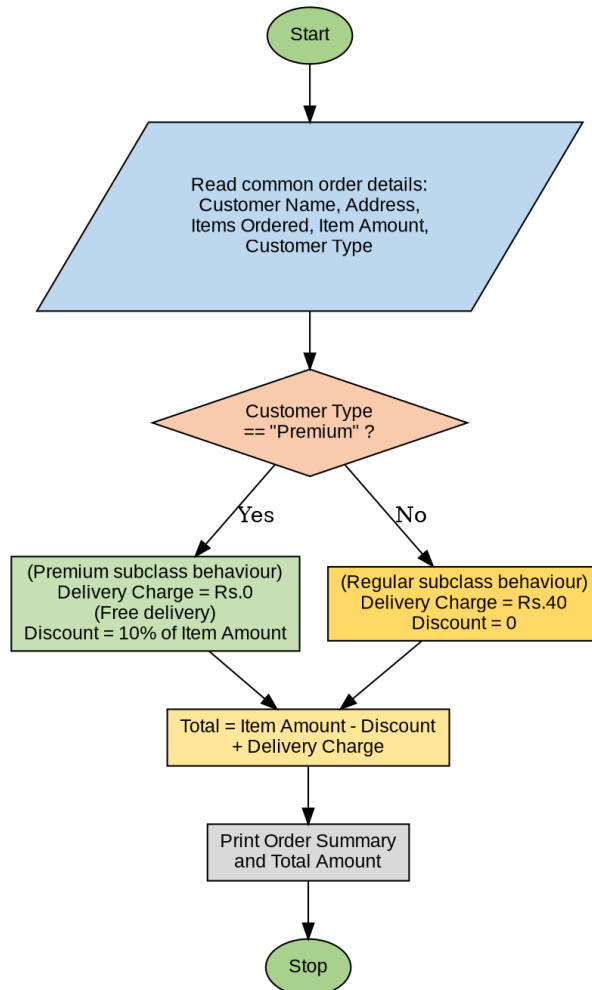


Figure 3.1: Flowchart for Food Order Billing (Regular / Premium Customer)

Pseudocode

```

BEGIN
  READ customerName, address, itemsOrdered, itemAmount, customerType

  IF customerType == "Premium" THEN
    deliveryCharge = 0      { free delivery for Premium }
    discount = 0.10 * itemAmount
  ELSE
    deliveryCharge = 40     { flat charge for Regular }
    discount = 0
  END IF

  total = itemAmount - discount + deliveryCharge
  PRINT customerName, itemsOrdered, deliveryCharge, discount, total
END
  
```

Relating the Design to Inheritance

The food-ordering problem naturally maps onto the Object-Oriented concept of inheritance. All customers share a common set of order details (name, address, items ordered, item amount), which forms a base/parent class. Regular and Premium customers only differ in how the delivery charge and discount are computed, which forms the specialised behaviour of derived/child classes.

OOP Element	Class	Contents
Base / Parent class	Customer	customerName, address, itemsOrdered, itemAmount
Derived class 1	RegularCustomer extends Customer	deliveryCharge = 40, discount = 0
Derived class 2	PremiumCustomer extends Customer	deliveryCharge = 0, discount = 10% of itemAmount

In Java terms, this can be implemented by writing a Customer class that holds the common order fields and a method computeTotal(), and then having RegularCustomer and PremiumCustomer classes that extend Customer and override (or supply values for) the delivery charge and discount. This avoids code duplication for the common order details, while still allowing each customer type to have its own billing rule — which is exactly the purpose of inheritance in Object-Oriented Programming.

Sample Java Skeleton Illustrating the Inheritance Relationship

```
class Customer {
    String name, address, itemsOrdered;
    double itemAmount;

    double computeTotal(double deliveryCharge, double discount) {
        return itemAmount - discount + deliveryCharge;
    }
}

class RegularCustomer extends Customer {
    double deliveryCharge = 40;
    double discount = 0;
}

class PremiumCustomer extends Customer {
    double deliveryCharge = 0;
    double discount = 0.10 * itemAmount;
}
```